

מערכת לניהול תורנויות חיילים

רקע וסיפור הפרויקט

במסגרת פרויקט קוד-קוד, היחידה מנהלת תורנויות שבועיות מגוונות: שמירה, מטבח, סיוור, ניקיון ועוד. כל שבוע צריך לשבץ חיילים לתורנויות שונות, לעקוב אחרי ביצוע התורנויות, ולדעת מי ביצע את התורנות שלו ומי פספס.

לאחרונה המערכת הישנה קרסה, וכל המידע אבד. אין דרך לדעת מי שובץ לאיזו תורנות, מי ביצע ומי פספס. המצב יצר בלגן גדול ביחידה.

המשימה שלך: לבנות מערכת חלופית, פשוטה אך אמינה, שתאפשר לנהל את כל התורנויות של החיילים ביחידה.

מה המערכת צריכה לעשות?

המערכת צריכה לאפשר למשתמש (המפקד או הממונה על התורנויות) לבצע את הפעולות הבאות:

ניהול חיילים

- **הוספת חייל חדש למערכת** - כל חייל מזהה על ידי מספר אישי ושם
- **הסרת חייל מהמערכת** - כאשר חייל משתחרר או עובר יחידה
- **צפייה ברשימת כל החיילים** - כדי לדעת מי רשום במערכת

ניהול תורנויות

- **הוספת תורנות לחייל** - שיבוץ חייל לתורנות מסוימת (למשל: "שמירה ליל שישי")
- **עדכון סטטוס תורנות** - סימון האם התורנות בוצעה, עדיין ממתנה, או פוספסה
- **צפייה בתורנויות של חייל** - לראות את כל התורנויות שמשויכות לחייל מסוים

המערכת צריכה להיות ידידותית למשתמש, עם תפריט ברור שמאפשר לבחור פעולה ולבצע אותה בקלות.

מבנה הפרויקט - קבצים ואחריותם

(Separation of Concerns) הפרויקט מחולק למספר קבצים, כאשר כל קובץ אחראי על תחום אחד בלבד. זה נקרא **הפרדת אחריות** (Separation of Concerns).

למה חשוב לחלק את הקוד לקבצים?

כאשר כל הקוד נמצא בקובץ אחד גדול, זה הופך להיות מבולגן וקשה להבנה. קשה למצוא איפה נמצאת פונקציה מסוימת, קשה לתקן באגים, וקשה להוסיף תכונות חדשות.

כאשר מחלקים את הקוד לקבצים לפי אחריות, כל קובץ עוסק בנושא אחד בלבד. זה עוזר:

- **להבין את הקוד** - יודעים בדיוק איפה לחפש דברים
- **לתחזק את הקוד** - קל יותר לתקן באגים
- **לעבוד בצוות** - כל אחד יכול לעבוד על קובץ אחר
- **להרחיב את המערכת** - קל להוסיף תכונות חדשות

הקבצים במערכת

נקודת הכניסה למערכת - `main.py`

אחריות: תפריט ראשי וניתוב

קובץ זה אחראי על:

- הצגת התפריט הראשי למשתמש
- קבלת בחירה מהמשתמש
- הפעלת הפעולה המתאימה לפי הבחירה

זהו הקובץ שמריץ את כל המערכת. הוא לא מכיל לוגיקה מורכבת - רק מציג אפשרויות ומפנה לקבצים האחרים.

למה זה חשוב? כך התפריט נמצא במקום אחד, ואם רוצים לשנות את התצוגה או להוסיף אפשרות חדשה, יודעים בדיוק איפה לעשות את זה.

אחסון הנתונים - `data.py`

אחריות: שמירת כל המידע של המערכת

קובץ זה מכיל את המידע על כל החיילים והתורנויות שלהם. כל המידע נשמר כאן במבנה מסודר.

למה זה חשוב? הפרדה בין הנתונים לבין הקוד שמשתמש בהם. אם בעתיד נרצה לשמור את המידע בקובץ או במסד נתונים, נצטרך לשנות רק את הקובץ הזה.

ניהול חיילים - `soldier_manager.py`

אחריות: ניהול המידע שקשור לחיילים

קובץ זה אחראי על:

- הוספת חייל חדש למערכת
- הסרת חייל מהמערכת
- שליפת רשימת החיילים

למה זה חשוב? כל הפעולות שקשורות לחיילים נמצאות במקום אחד. אם צריך לשנות איך מוסיפים חייל, יודעים בדיוק איפה לחפש.

ניהול תורנויות - `duty_manager.py`

אחריות: ניהול המידע שקשור לתורנויות

קובץ זה אחראי על:

- הוספת תורנות לחייל
- עדכון סטטוס של תורנות (האם בוצעה, ממתינה, או פוספסה)
- שליפת התורנויות של חייל מסוים

למה זה חשוב? הפרדה ברורה בין ניהול חיילים לניהול תורנויות. כל אחד מהם הוא תחום נפרד עם כללים משלו.

פונקציות עזר - `utils.py`

אחריות: פונקציות שימושיות שחוזרות על עצמן

קובץ זה מכיל פונקציות עזר שמשמשות את הקבצים האחרים. למשל, פונקציה שמחפשת חייל לפי מספר אישי, או פונקציה שבודקת אם סטטוס תקין.

למה זה חשוב? במקום לכתוב את אותו קוד כמה פעמים בקבצים שונים, כותבים אותו פעם אחת בקובץ העזר **DRY** (Don't Repeat Yourself) ומשתמשים בו מכל מקום. זה נקרא

מבני הנתונים במערכת

כדי שהמערכת תוכל לשמור ולנהל מידע על חיילים ותורנויות, צריך להחליט איך לייצג את המידע הזה. במערכת שלנו, (lists) ו**רשימות** (dictionaries) **מילונים** Python נשתמש במבני נתונים של

מבנה נתונים של חייל

כל חייל במערכת מיוצג על ידי **מילון** שמכיל את השדות הבאים:

- **id** (טיפוס: `int`) מספר אישי ייחודי שמזהה את החייל - `id`
- **name** (טיפוס: `str`) שם החייל - `name`
- **duties** (טיפוס: `list`) רשימה של כל התורנויות שמשובצות לחייל - `duties`

?**שאלה** מדוע לא לזהות את החייל לצורך חיפוש או מחיקה עלפי שמו בלבד?

דוגמה (זו רק דוגמה להמחשה)

```
{
  "id": 12345,
  "name": "דוד כהן",
  "duties": [...]
}
```

הוא המספר האישי של החייל. זהו מזהה ייחודי שמשמש לחיפוש, הוספה והסרה של חיילים **id חשוב**: השדה

מבנה נתונים של תורנות

כל תורנות מיוצגת על ידי **מילון** שמכיל את השדות הבאים:

- **name** (טיפוס: `str`) שם התורנות - "למשל: "שמירה", "מטבח", "סיור", `str` (טיפוס: `str`) שם התורנות - `name`
- **day** (טיפוס: `str`) יום בשבוע - "sunday", "monday", "tuesday", "wednesday", "thursday"
- **status** (טיפוס: `str`) מצב התורנות - "pending", "completed", או "missed"

דוגמה (זו רק דוגמה להמחשה)

```
{  
  "name": "guard duty",  
  "day": "sunday",  
  "status": "pending"  
}
```

איך זה עובד ביחד?

המערכת שומרת **רשימה של חיילים**. כל חייל ברשימה הוא מילון, וכל מילון של חייל מכיל רשימה של תורנויות. כל תורנות היא מילון.

מילונים בתוך רשימות בתוך מילונים - (nested data structure) זה נקרא **מבנה נתונים מקונן**.

למה חשוב להבין את זה?

- כדי לדעת איך לגשת למידע (למשל, איך למצוא את התורנויות של חייל מסוים)
- כדי לדעת איך להוסיף מידע חדש (למשל, איך להוסיף תורנות לחייל)
- כדי לדעת איך לעדכן מידע (למשל, איך לשנות סטטוס של תורנות)

חוקי המערכת - התנהגות חובה

המערכת חייבת לפעול לפי חוקים ברורים. אלו לא המלצות - אלו דרישות קשיחות שהמערכת חייבת לעמוד בהן:

חוקים לגבי חיילים

1. ייחודי (מספר אישי) **id**

- שונה **id** כל חייל חייב להיות עם
- שכבר קיים במערכת **id** אי אפשר להוסיף חייל עם
- זה מבטיח שלא יהיו שני חיילים עם אותו מספר אישי

2. **name** לא יכול להיות ריק

- כל חייל חייב להיות עם שם
- ריק **name** או עם **name** אי אפשר להוסיף חייל בלי

3. הסרת חייל

- (**id** לפי) אי אפשר להסיר חייל שלא קיים

חוקים לגבי תורנויות

4. תורנות רק לחיילים קיימים

- (**id** לפי) אפשר להוסיף תורנות רק לחייל שכבר רשום במערכת

5. אין תורנויות כפולות

- **name** חייל לא יכול להיות עם שתי תורנויות עם אותו

- שמירה ליל שישי, אי אפשר להוסיף לו עוד תורנות עם `name` למשל, אם לחייל כבר יש תורנות עם אותו שם

6. `status` בלבד חוקי

- `status`: כל תורנות חייבת להיות עם אחד מהערכים הבאים בשדה:
 - `"pending"` - ממתינה (התורנות עדיין לא התבצעה)
 - `"completed"` - בוצעה (החייל ביצע את התורנות)
 - `"missed"` - פוספסה (החייל לא ביצע את התורנות)
- אחר `status` אי אפשר להגדיר

7. `day` בלבד חוקי

- `day`: כל תורנות חייבת להיות עם אחד מהערכים הבאים בשדה:
 - `"sunday"` - יום ראשון
 - `"monday"` - יום שני
 - `"tuesday"` - יום שלישי
 - `"wednesday"` - יום רביעי
 - `"thursday"` - יום חמישי
- `"saturday"` או שבת (`"friday"`) **אסור** להוסיף תורנויות ביום שישי
- אחר `day` אי אפשר להגדיר

Exceptions) טיפול בחרגות

בצורה נכונה ועקבית לאורך כל הפרויקט **exceptions**-המערכת חייבת להשתמש ב

Exceptions-עקרונות שימוש ב

raise-מתי להשתמש ב:

- `soldier_manager.py`, `duty_manager.py`, `utils.py` (בקבצים) בפונקציות לוגיקה עסקית
- כאשר מתגלה מצב שגוי או לא חוקי
- מתאים exception זרקו, `None` או `False` במקום להחזיר

try-except-מתי להשתמש ב:

- `main.py` ב-`handle_*` בפונקציות ה
- שנזרקו מהלוגיקה העסקית exceptions כדי לתפוס
- כדי להציג הודעת שגיאה ברורה למשתמש
- כדי למנוע קריסת התוכנית

דוגמאות לשימוש

exception: בלוגיקה עסקית - זריקת

```
def add_soldier(soldier_id: int, name: str):
    if find_soldier_by_id(soldier_id):
        raise ValueError(f"ID {soldier_id} חיייל עם")
```

```
if not is_valid_name(name):
    raise ValueError("שם חייל לא יכול להיות ריק")

# המשיך הלוגיקה...
```

2-main.py - exception תפיסת:

```
def handle_add_soldier():
    soldier_id = int(input("הכנס מספר אישי: "))
    name = input("הכנס שם: ")

    try:
        add_soldier(soldier_id, name)
        print("✓ חייל נוסף בהצלחה")
    except ValueError as e:
        print(f"X שגיאה: {e}")
```

מומלצים Exceptions סוגי

- **ValueError** - (אסור day, לא חוקי status, ריק name, כפול id) עבור ערכים לא תקינים
- **KeyError** - עבור מקרים שבהם פריט לא נמצא (חייל לא קיים, תורנות לא נמצאה)

דרישות חובה

1. במקרה של שגיאה exception **כל פונקציה בלוגיקה עסקית** חייבת לזרוק.
2. ולהציג הודעה ברורה exceptions חייבת לתפוס **handle_* כל פונקציית**
3. **הודעות שגיאה** חייבות להיות באנגלית וברורות למשתמש
4. נתפס ומטופל exception **התוכנית לעולם לא תקרוס** - כל

תהליך עבודה - שלבי התכנון והמימוש

העבודה על הפרויקט מתבצעת ב-3 שלבים עם אישור מדריך בין כל שלב.

שלב 1: תכנון ראשוני (בכתב יד)

מה צריך להכין

1. תרשים זרימה ראשוני

- בכתב יד על נייר
- מבט על על הזרימה של התוכנית (אין צורך לתרשים נפרד של הפונקציות הפנימיות)
- כולל נקודות החלטה עיקריות

2. מסמך פונקציות

- בכתב יד על נייר
- רשימה של כל הפונקציות שתצטרכו

- לכל פונקציה:
 - שם הפונקציה
 - תפקיד הפונקציה (מה היא עושה)
 - קלט (מה היא מקבלת)
 - פלט (מה היא מחזירה)
 - אילו שגיאות היא עלולה להחזיר
- **לא חייב להיות בפורמט קוד** - תיאור מילולי מספיק

הגשה: הגשת המסמכים בכתב יד למדריך

רק לאחר אישור המדריך ניתן לעבור לשלב 2 ⚠

שלב 2: תכנון דיגיטלי

מה צריך להכין:

1. draw.io-תרשים זרימה ב

- draw.io-יצירת תרשים זרימה מפורט ב
- על בסיס התרשים הראשוני שאושר
- כולל את כל הפרטים והזרימות

2. (.py) קובצי חתימות פונקציות

- חתימות פונקציות בלבד
- על בסיס מסמך הפונקציות שאושר
- בגוף הפונקציות **pass** ללא מימוש - רק
- type hints ו-docstrings כולל

דוגמה לפורמט:

```
def add_soldier(soldier_id: int, name: str) -> None:
    """
    מוסיפה חייל חדש למערכת

    קלט:
        soldier_id: מספר אישי של החייל
        name: שם החייל

    פלט: None

    ריק name כבר קיים או id אם ValueError: זורקת
    """
    pass
```

למדריך .py וקובץ draw.io **הגשה:** הגשת קובץ

רק לאחר אישור המדריך ניתן לעבור לשלב 3 ⚠

שלב 3: מימוש הקוד

לאחר אישור שלב 2, המדריכים יספקו:

- תרשים זרימה מלא ומפורט
- קובץ חתימות פונקציות מלא

המשימה שלכם:

- לממש את הקוד על פי תרשים הזרימה שסופק
- לממש את הפונקציות על פי החתימות שסופקו
- להקפיד על כל החוקים והדרישות שפורטו במסמך זה
- בצורה נכונה exceptions-להשתמש ב
- לוודא שהמערכת עובדת כמצופה

סיכום תהליך העבודה

```
שלב 1 (כתב יד)
↓
אישור מדריך
↓
שלב 2 (דיגיטלי)
↓
אישור מדריך
↓
שלב 3 (מימוש)
↓
הגשה סופית
```

חשוב: אין לדלג על שלבים. כל שלב חייב לקבל אישור לפני המעבר הבא.

הגשה סופית

מה צריך להגיש?

ייעודי לפרויקט GitHub repository-כל העבודה חייבת להיות ב

הרפוזיטורי חייב לכלול:

1. המסמכים שיצרתם:

- draw.io-קובץ תרשים הזרימה מ
- (.py) קובץ חתימות הפונקציות שיצרתם

2. המסמכים שקיבלתם מהמדריכים:

- תרשים הזרימה המלא שסופק
- קובץ חתימות הפונקציות המלא שסופק

3. הקוד הסופי:

- `main.py`
- `data.py`
- `soldier_manager.py`
- `duty_manager.py`
- `utils.py`
- כל קובץ נוסף שיצרתם

Git דרישות עבודה עם

Git! **חשוב מאוד:** תיבחנו גם על עבודה נכונה עם

דרישות:

- **קומיטים קטנים ותכופים** - לא קומיט אחד גדול בסוף
- **הודעות קומיט באנגלית בלבד** - תיאור מדויק של השינוי
- **קומיט אחרי כל שלב משמעותי** - למשל "Add add_soldier function", "Fix validation in duty_manager"

דוגמאות להודעות קומיט טובות:

- ✓ "Add add_soldier function with validation"
- ✓ "Implement main menu in main.py"
- ✓ "Fix bug in friday/saturday validation"
- ✓ "Add exception handling in handle_add_duty"

דוגמאות להודעות קומיט לא טובות:

- X "update"
- X "fixes"
- X "done"
- X "123"

שלכם מראה את תהליך העבודה שלכם ואת החשיבה שלכם-Git **זכרו:** היסטוריית ה